KV-Cache Quantization on NVIDIA Jetson Orin Nano

Energy, speed, memory and accuracy of f16, 8-bit and 4-bit KV caches. Revised report (v2): addresses the review of the v1 report and adds seven new experiments.

Koushik Bhupathiraju · Supervisors: Prof. Taskin Kocak, Prof. Rongyu Lin

v1 experiments: 19 October 2026 · v2 experiments: 26–27 October 2026

Code and raw data: github.com/koushikvarmaJS/thesis2

Contents

0. Response to the review
1. Summary and main claim
2. Background and thesis scope
3. What happened, step by step
4. Hardware
5. Software, libraries and tools
6. Models
7. KV cache formats
8. How energy was measured
9. Experiments and parameters
10. Results
11. Data quality
12. Scope and limitations
13. Next steps
Appendix A. Reproduction · B. Files · C. v1 per-round values · D. Glossary

0. Response to the review

Each point of Prof. Lin's review, what was done about it, and where the answer is. All v2 numbers come from new runs on 26–27 October; the v1 numbers are only quoted for comparison.

Review point	What was done	Where
Flash-attention-off baseline: was f16 held back by forcing FA on?	f16 run with FA on and off in every v2 grid (5 rounds). FA off is much worse: +227.6% (Llama) and +270.4% (Qwen) energy per request at 16K. FA on is the stronger baseline, so v1 compared against the right one.	10.3
Gross vs net energy; the 7W/15W comparison should show both	Every table now has net (idle subtracted) and gross. Idle is measured at the start and end of every set.	8, 10.1, 10.11
CPU fallback? Per-rail energy	CPU busy and GPU load are now sampled with the power. In decode the CPU is 2–3% busy for every format and the GPU 90–97%, so nothing falls back to the CPU. The extra energy of 8-bit/4-bit is almost all on the CPU+GPU rail; the SoC rail barely changes.	10.5, 10.6
Prefill and decode measured separately (llama-bench)	New set B: pp and tg as separate llama-bench tests with energy per repetition. tg128 at 16K agrees with the server timestamp split within a few percent.	10.4
Concurrency (np = 2/4/8)	New set D: 1, 2, 4, 8 parallel sessions at 4K and 16K per session. The penalty stays under batching; f16 with 8 × 16K does not fit on Llama, 8-bit and 4-bit do.	10.7
4-bit weights (Q4_K_M)	New set E: both models with Q4_K_M weights, full context grid, 5 rounds. Same result as with Q8_0 weights.	10.8
At least 5 paired rounds, CIs	Main grid, weights set and llama-bench: 5 rounds; concurrency 3 rounds. All changes are paired by round with 95% CIs; (n.s.) marks a CI that includes zero.	8, 11
Is the 7W Qwen 16K cell an outlier?	No: the 3 v1 rounds are within 1% of each other (Appendix C), and the 7W rerun in v2 confirms it.	App. C
Power column did not match J ÷ s on slide 13	v1 showed whole-request power; the v2 tables show decode-window power, which matches decode J ÷ s.	8, 10.5
Accuracy at long context; is 4-bit K or V the problem?	Perplexity at 8K for both models (16K runs out of memory in llama-perplexity), plus a K/V split for Qwen and Llama with the all-quants build: 4-bit keys, not values, break Qwen.	10.9
Why is it slower? Mechanism	Nsight Systems kernel profile at 16K: with an 8-bit or 4-bit cache, decode attention moves from a tensor-core kernel to a CUDA-core "vec" kernel that takes +130% longer per call on Llama (4-bit). Nsight Compute: the f16 kernel is memory-bound (88% of peak memory, 10% compute); the 4-bit kernel is compute-heavy (74% compute) and re-reads the cache for every query head.	10.5
Where does compression help? Memory fit test	New set G: Llama-1B at 32K/64K and Llama-3B at 16K/32K/64K, auto-fit off. On the 3B model f16 fits only 16K, 8-bit 32K and 4-bit 64K.	10.10
Thesis main question and a falsifiable claim	Proposed in section 1 (please confirm).	1
Public repository with raw data	github.com/koushikvarmaJS/thesis2: code, every raw JSONL/CSV, analysis scripts and this PDF's generator.	App. B

1. Summary and main claim

Proposed thesis main question. On a memory-limited edge device, does compressing the KV cache reduce the energy of LLM inference, and if not, what does it buy?

Proposed falsifiable claim. On a Jetson Orin Nano running llama.cpp, an 8-bit or 4-bit KV cache uses **more** energy per request than an f16 cache at every context length from 1K to 16K tokens, and the gap grows with context. The cause is compute, not memory traffic: the quantized cache moves decode attention off the tensor-core kernel. Compression only pays as capacity: it lets contexts and session counts run that do not fit with f16. The claim is refuted by any paired cell where a quantized cache is significantly cheaper than f16, or by f16 fitting everything the quantized caches fit.

At 16K tokens, 15W	Llama-1B 8-bit	Llama-1B 4-bit	Qwen-0.5B 8-bit	Qwen-0.5B 4-bit
Energy per request, net	+15.9%	+12.6%	+18.0%	+16.0%
Energy per request, net, 7W	+20.7%	+17.6%	+27.9%	+20.6%
Decode energy per token, net	+35.9%	+28.6%	+58.0%	+50.4%
Decode speed	-19.9%	-22.2%	-33.3%	-36.2%
KV cache memory	-47%	-72%	-47%	-72%

• Every quantized cell in the 15W main grid uses more energy than f16, and every change is significant (paired, 5 rounds). At 1K the penalty is 2.6–4.6%; at 16K it is 13–18% per request and 29–58% per decoded token.

• The f16 baseline was not held back: turning flash attention off makes f16 3.3–3.7× more expensive per request at 16K.

• Nothing runs on the CPU. The extra energy is GPU work: decode attention on an 8-bit/4-bit cache uses a "vec" kernel without tensor cores, 2.1–2.3× slower per call than f16 on Llama and 3.5–3.9× on Qwen (whose 7 query heads per KV head make the f16 tensor-core kernel especially efficient). Decode power barely changes; time grows, and energy with it.

• The same holds with 4-bit weights and with up to 8 concurrent sessions. What compression buys is room: Llama-1B cannot hold 8 sessions × 16K tokens in f16 (4.1 GiB of cache) but can in 8-bit or 4-bit.

• At 7W the result is the same: all 48 quantized cells are significantly more expensive than f16 (+2.6% at 1K up to +28% per request at 16K). 7W itself is the more expensive mode: +21–31% net and +60–73% gross energy per request against 15W (section 10.11).

2. Background and thesis scope

The thesis (spring 2026) measured the energy of KV-cache *eviction* methods (StreamingLLM and SnapKV through NVIDIA's kvpress library) on a Tesla T4 cloud GPU, with Qwen2.5-0.5B and Llama-3.2-1B. The paper extends it to an edge device, the Jetson Orin Nano, where memory is the limiting resource.

This report covers the first edge method family: KV-cache *quantization* (storing K and V in 8 or 4 bits) in llama.cpp. Eviction on the Jetson needs PyTorch and kvpress and comes next (section 13).

Correction to v1. v1 said quantized KV caches could not run on the T4. More precisely: the published KIVI implementations failed there. optimum-quanto's Marlin kernel and KIVI's own CUDA extension need an Ampere GPU (sm_80 or newer; the T4 is sm_75), and the hqq backend failed to install. llama.cpp's quantized KV cache would have built for the T4. The move to the Jetson is about the edge question, not a T4 limit.

Why perplexity is meaningful here. In the thesis, kvpress compressed the cache only after the prompt was read, so perplexity over the prompt never saw the compression. In llama.cpp every token's K and V are written to the quantized cache immediately and attention reads them back from it, so perplexity includes the quantization error.

Related work. Server-GPU work (QServe, BitDecoding, KVmix and others) reports that low-bit KV caches make decode attention compute-bound because of dequantization, and builds special kernels to hide it. No measurement of KV quantization energy on an edge device was found; that is this paper's contribution.

3. What happened, step by step

When	What
19 Oct	v1: llama.cpp built on the Jetson; 3 rounds × 2 models × 3 formats × 4 contexts at 15W, then 7W; perplexity at 2K. Results deck and report (v1).
21–25 Oct	Review by Prof. Lin. v2 code: K/V pairs and FA on/off, CPU/GPU load sampling, concurrency, llama-bench wrapper, perplexity at 16K, profiling, fit test.
26 Oct	Found the desktop's Software Updater using a full CPU core after every boot (raised idle power in v1 by about 0.3–0.5 W). It is closed before every v2 set.
10-26 21:12 (15W)	A main grid: f16 (flash attention on and off), q8_0, q4_0; 5 rounds
10-26 21:52 (15W)	B llama-bench: prompt and generation measured separately
10-26 22:19 (15W)	D concurrency: 1, 2, 4, 8 sessions
10-26 23:41 (15W)	E 4-bit weights (Q4_K_M)
10-27 00:05 (15W)	F nsys kernel profile at 16K
10-27 00:09 (15W)	C perplexity at 16K and Qwen K/V split (finishes the all-quants build first)
10-27 00:47 (15W)	G fit test: long contexts and a 3B model (f16 may not fit)
10-27 01:39 (15W)	C (rerun) perplexity at 8K and the K/V split at 8K: 16K ran out of memory (section 10.9)
10-27 02:16 (15W)	Check: Llama-1B f16 with 8 sessions × 16K; out of memory, board rebooted
10-27 02:20 (15W)	F (ncu) Nsight Compute counters of the decode attention kernel (root)
10-27 02:29 (7W)	A main grid: f16 (flash attention on and off), q8_0, q4_0; 5 rounds
10-27 03:54 (7W)	B llama-bench: prompt and generation measured separately

Times from results/run_v2_*.log. Each set's raw data is a separate file (Appendix B).

4. Hardware

Component	Detail
Board	NVIDIA Jetson Orin Nano Developer Kit (8 GB), hostname "ranjotsingh", runs as user koushik
CPU	6 × Arm Cortex-A78AE (4 online in 7W mode)
GPU	NVIDIA Ampere, compute capability 8.7 (sm_87), 1024 CUDA cores, 32 tensor cores
Memory	8 GB LPDDR5 shared by CPU and GPU; 7,486 MB visible to Linux; no swap
Storage	microSD card, 113 GB root partition
Power sensor	Onboard TI INA3221, three channels: VDD_IN (whole module), VDD_CPU_GPU_CV (CPU, GPU and vision accelerators on one rail), VDD_SOC
Cooling	Stock dev-kit heatsink and fan
Control	SSH over Wi-Fi from a MacBook (Apple M2); long runs inside tmux

Power mode	nvpmodel ID	GPU max clock	CPU cores	Idle VDD_IN, v1	Idle VDD_IN, v2
15W (default)	0	624.75 MHz	6	3.57 W	3.08 W (start) / 3.34 W (end)
7W	1	408 MHz	4	3.41 W	3.68 W (start) / 3.03 W (end)

Only these two modes exist in /etc/nvpmodel.conf on this image (no 25W / MAXN Super). Switching needs a reboot. v2 idle power is lower than v1 because the desktop's Software Updater, which starts after every boot and kept one CPU core busy, was closed before every experiment set (section 9).

5. Software, libraries and tools

Item	Version / setting
Operating system	Ubuntu 24.04.4 LTS, Linux 6.8.12-1021-tegra
Jetson Linux	L4T R39.2.0 (JetPack 7 line)
CUDA	CUDA Toolkit 13.2.2 (nvcc 13.2.86), cuBLAS 13.4.1
Build tools	GCC 13.3.0, CMake 3.28.3
Inference engine	llama.cpp commit 7adf160 (0.7.0-dev). Main build: -DGGML_CUDA=ON -DCMAKE_CUDA_ARCHITECTURES=87, Release. Used for all energy runs, v1 and v2.
Second build	Same commit and flags plus -DGGML_CUDA_FA_ALL_QUANTS=ON. Without it, GPU flash attention only exists for matching K/V types (f16/f16, q8_0/q8_0, q4_0/q4_0); a mixed pair such as K f16 + V q4_0 would run attention on the CPU. Used only for the K/V split perplexity tests.
llama.cpp programs	llama-server (energy), llama-bench (separate prefill/decode), llama-perplexity (accuracy)
Profilers	NVIDIA Nsight Systems 2025.6.3 (nsys), Nsight Compute 2026.1.1 (ncu)
Benchmark code	Python 3.12.3, standard library only: power.py, bench_kv.py, bench_pp_tg.py, ppl_v2.sh, profile.sh, run_v2.sh
Analysis (Mac)	Python 3.14 + matplotlib 3.11.2 (analyze_v2.py), reportlab 5.1.0 (this PDF)

6. Models

Property	Llama-3.2-1B-Instruct	Qwen2.5-0.5B-Instruct	Llama-3.2-3B-Instruct
Used in	all experiments	all except fit test	fit test only
Weights (main)	Q8_0, 1.32 GB (bartowski GGUF)	Q8_0, 0.68 GB (Qwen GGUF)	–
Weights (4-bit set)	Q4_K_M, 0.81 GB	Q4_K_M, 0.49 GB	Q4_K_M, 2.02 GB
Layers	16	24	28
Query heads / KV heads	32 / 8	14 / 2	24 / 8
Head size	64	64	128
Trained context	131,072	32,768	131,072
KV cache per token, f16	32 KiB	12 KiB	112 KiB

Values read from each GGUF header. KV bytes per token = layers × KV heads × head size × 2 (K and V) × bytes per value.

7. KV cache formats

Format	How values are stored	Bytes / value	Size vs f16
f16	16-bit floating point, no compression	2	100%
q8_0	blocks of 32: 32 × int8 + one fp16 scale (34 bytes); symmetric, no offset	1.0625	53%
q4_0	blocks of 32: 32 × 4-bit + one fp16 scale (18 bytes); symmetric, no offset	0.5625	28%
q4_1	blocks of 32: 32 × 4-bit + fp16 scale + fp16 minimum (20 bytes); has an offset	0.625	31%

In q4_0 the scale is the block value with the largest magnitude divided by −8, so that value is stored exactly and the rest are rounded to 16 levels around zero. q4_1 stores a minimum as well, so a block whose values all sit far from zero (an offset) still gets its 16 levels where the values are. In the energy runs K and V always use the same format. Flash attention is on in all runs except the "f16, flash attn off" baseline (section 10.3).

8. How energy was measured

• **Sensor and sampling.** A background thread reads voltage and current of the three INA3221 rails from sysfs every 50 ms (about 18 samples/s); power = mV × mA / 10^6. Each sample also stores available memory, the GPU load (0–100%, from the GPU's sysfs *load* file) and the CPU busy time (from /proc/stat). Energy over any window is the trapezoid integral of the samples.

• **Gross and net energy.** Gross energy is what the module actually drew. Net energy subtracts the idle power (10 s with nothing running, measured before and after every experiment set) times the duration. Both are reported: net isolates the work, gross is what a battery pays. The difference matters most when run times differ a lot, as between 7W and 15W.

• **Per request.** Energy of one request = from sending it to receiving the reply. This is the headline metric in v2.

• **Decode energy per token (server runs).** Prefill is taken to end at send time + llama.cpp's own prompt time; the rest is decode. Decode J/token = decode energy ÷ 128. This split is approximate, so v2 also measures prefill and decode as separate llama-bench runs (below).

• **Decode energy per token (llama-bench).** llama-bench runs prompt processing (pp) and generation (tg) as separate tests. With --progress it prints a line just before each timed repetition and reports each repetition's duration; the energy of a repetition is integrated from that line's timestamp over that duration. "tg128 at depth 16,384" first fills the cache with 16,384 tokens (untimed), then times 128 generated tokens: pure decode energy, no split needed.

• **Power columns.** "Decode power" is the mean gross VDD_IN power inside the decode window only. v1's slide 13 showed the mean over the whole request (prefill + decode), which is lower because prefill draws less power; that is why it did not match decode joules ÷ decode time.

• **Concurrent sessions.** With N sessions the requests overlap. Energy per generated token = request energy ÷ (N × 128). Decode J/token uses the window in which all N sessions are decoding (from the last session to start decoding until the first to finish), divided by the tokens generated in it.

• **Statistics.** Every cell is a mean with a 95% confidence interval (Student t). Changes against f16 are paired: each format is compared with f16 from the same round (minutes apart, same thermal state) and the CI is taken over the per-round ratios. A change whose CI includes zero is marked (n.s.), not separable from zero.

9. Experiments and parameters

Set	What	Models	Formats	Contexts	Rounds
A	Main grid, llama-server, 128 generated tokens	Llama-1B, Qwen-0.5B (Q8_0 weights)	f16 (FA on), f16 (FA off), q8_0, q4_0	1K, 4K, 8K, 16K	5
B	llama-bench: pp at 1K/16K, tg128 at depth 1K/16K	same	same	–	5 reps
C	Perplexity, WikiText-2; K/V split	same	f16, q8_0, q4_0, q4_1, mixed K/V	2K, 8K	1 (± error)
D	Concurrency: N sessions sent at once	same	f16, q8_0, q4_0	4K, 16K per session; N = 1, 2, 4, 8	3
E	Main grid with 4-bit weights	Llama-1B, Qwen-0.5B (Q4_K_M)	f16, q8_0, q4_0	1K–16K	5
F	Kernel profile (nsys), attention counters (ncu)	Llama-1B, Qwen-0.5B	f16 (FA on/off), q8_0, q4_0	depth 16K + 64 tokens	1
G	Fit test: largest context per format	Llama-1B, Llama-3B (Q4_K_M)	f16, q8_0, q4_0	1B: 32K/64K; 3B: 16K/32K/64K	2

• **Common settings.** All layers on the GPU (-ngl 99), flash attention on unless stated, -c = context × sessions, one server start per format and round. The prompt is WikiText-2 test text cut to context − 128 − 16 tokens, sent as token IDs; ignore_eos makes every reply exactly 128 tokens; cache_prompt=false makes every request process its prompt. Temperature 0.

• **Order.** Inside each round, formats and contexts run in a shuffled order (fixed seed), so no format always runs first or on a cooler board. A 1K-token warm-up request runs after each server start.

• **Sets A and B run at 15W and at 7W**; C–G at 15W only (the 7W slowdown affects all formats alike in v1).

• **Before every set** the Software Updater and GNOME Software are closed and idle power is measured for 10 s.

10. Results

10.1 Headline: energy per request

One request = a prompt of the given length plus exactly 128 generated tokens, the whole thing from send to reply. Change against f16 (flash attention on), paired by round, mean ± 95% CI over 5 rounds. Positive = the compressed cache uses **more** energy. (n.s.) = the CI includes zero.

15W, net energy (idle subtracted)

Context	Llama-1B 8-bit	Llama-1B 4-bit	Qwen-0.5B 8-bit	Qwen-0.5B 4-bit
1K	+4.3% ± 2.8	+2.6% ± 1.1	+4.6% ± 1.3	+4.4% ± 1.0
4K	+6.7% ± 0.7	+5.9% ± 1.4	+10.9% ± 1.3	+10.3% ± 0.9
8K	+12.0% ± 3.3	+9.3% ± 0.3	+16.8% ± 0.7	+13.5% ± 0.4
16K	+15.9% ± 3.0	+12.6% ± 0.5	+18.0% ± 1.0	+16.0% ± 0.9

15W, gross energy (what the module drew)

Context	Llama-1B 8-bit	Llama-1B 4-bit	Qwen-0.5B 8-bit	Qwen-0.5B 4-bit
1K	+4.2% ± 2.3	+2.8% ± 1.5	+4.9% ± 1.5	+4.9% ± 1.4
4K	+6.4% ± 0.8	+6.1% ± 1.9	+11.0% ± 1.9	+10.9% ± 1.3
8K	+10.8% ± 2.4	+9.4% ± 0.5	+16.3% ± 0.7	+14.5% ± 0.3
16K	+14.0% ± 2.1	+12.5% ± 0.2	+17.0% ± 0.6	+16.4% ± 0.6

7W, net energy (idle subtracted)

Context	Llama-1B 8-bit	Llama-1B 4-bit	Qwen-0.5B 8-bit	Qwen-0.5B 4-bit
1K	+3.2% ± 1.2	+2.6% ± 0.7	+5.4% ± 0.8	+4.7% ± 0.7
4K	+8.0% ± 0.6	+6.8% ± 0.6	+11.6% ± 0.5	+11.3% ± 0.6
8K	+15.8% ± 0.7	+13.1% ± 0.7	+21.0% ± 0.9	+14.2% ± 0.6
16K	+20.7% ± 1.6	+17.6% ± 0.4	+27.9% ± 0.3	+20.6% ± 0.3

7W, gross energy (what the module drew)

Context	Llama-1B 8-bit	Llama-1B 4-bit	Qwen-0.5B 8-bit	Qwen-0.5B 4-bit
1K	+3.9% ± 1.3	+3.5% ± 0.6	+6.9% ± 1.1	+7.0% ± 1.0
4K	+8.3% ± 0.7	+8.4% ± 0.6	+13.3% ± 0.5	+14.2% ± 0.6
8K	+13.8% ± 0.7	+13.6% ± 0.6	+19.7% ± 0.7	+17.6% ± 0.6
16K	+16.9% ± 0.8	+17.0% ± 0.2	+23.3% ± 0.2	+21.2% ± 0.1

f16 baseline, J per request (net / gross)	1K	4K	8K	16K
Llama-1B, 15W	33.9 / 50.0	48.0 / 70.7	69.5 / 101.8	123.8 / 180.2
Qwen-0.5B, 15W	13.0 / 20.8	20.6 / 31.8	32.8 / 49.5	66.0 / 97.1
Llama-1B, 7W	43.0 / 81.0	59.5 / 114.4	86.1 / 166.9	154.3 / 300.0
Qwen-0.5B, 7W	16.8 / 33.1	25.4 / 50.5	39.8 / 79.1	79.8 / 156.5

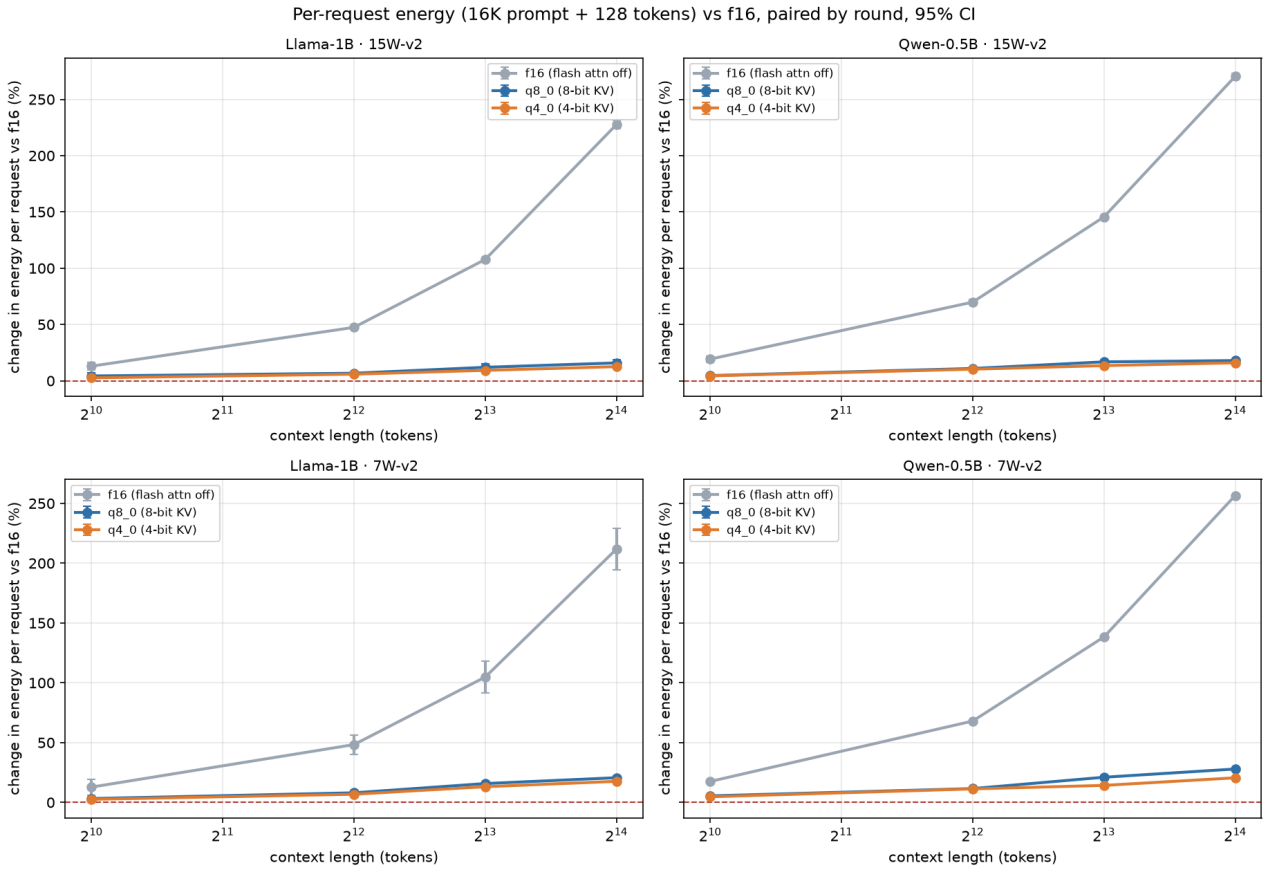


Figure 1. Change in energy per request against f16, paired by round, 95% CI. Hollow markers: not separable from zero. File: plots_v2/01_request_energy_change.png

10.2 Decode energy per generated token

The same runs, counting only the decode part (energy after the prompt has been read, ÷ 128). This is the v1 headline metric. It isolates the cost of reading the cache for each new token, so the changes are larger than per request, where prefill dilutes them.

15W, decode J/token, net

Context	Llama-1B 8-bit	Llama-1B 4-bit	Qwen-0.5B 8-bit	Qwen-0.5B 4-bit
1K	+4.2% ± 3.3	+2.2% ± 2.1	+3.8% ± 1.8	+3.9% ± 1.7
4K	+6.6% ± 2.2	+5.8% ± 4.2	+12.4% ± 4.9	+11.7% ± 3.6
8K	+18.2% ± 7.1	+13.5% ± 1.9	+33.4% ± 2.1	+25.2% ± 0.6
16K	+35.9% ± 2.5	+28.6% ± 0.8	+58.0% ± 3.3	+50.4% ± 3.7

7W, decode J/token, net

Context	Llama-1B 8-bit	Llama-1B 4-bit	Qwen-0.5B 8-bit	Qwen-0.5B 4-bit
1K	+3.6% ± 1.5	+2.8% ± 0.6	+5.4% ± 1.2	+4.8% ± 1.6
4K	+9.6% ± 1.0	+8.1% ± 0.8	+15.7% ± 1.5	+15.6% ± 1.7
8K	+27.4% ± 1.4	+22.3% ± 1.3	+43.9% ± 2.5	+27.9% ± 1.9
16K	+55.7% ± 0.4	+47.1% ± 0.2	+106.0% ± 1.6	+74.8% ± 1.1

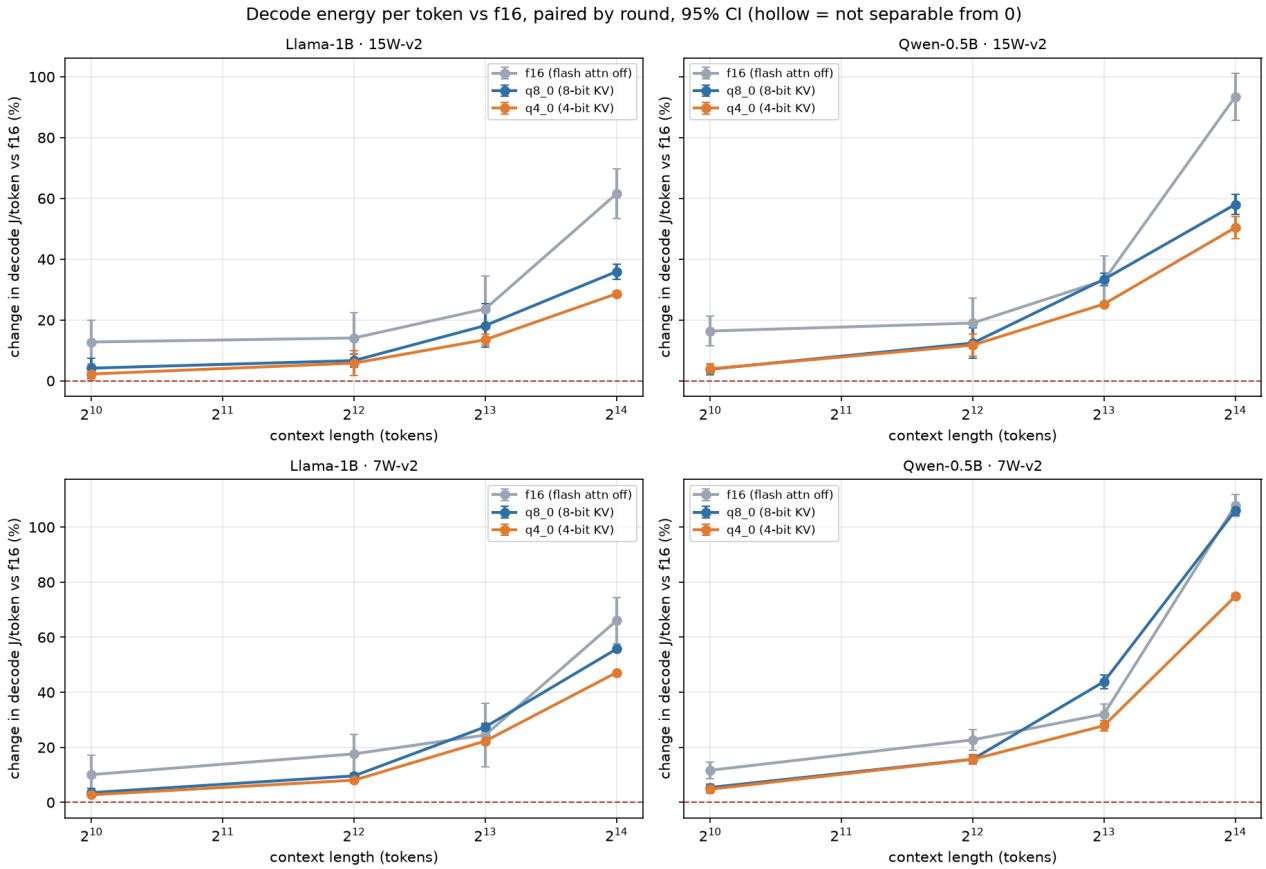


Figure 2. Change in decode energy per token against f16, paired by round, 95% CI. File: plots_v2/02_decode_energy_change.png

10.3 Baseline strength: f16 with flash attention off

v1 forced flash attention on for f16 too, because a quantized V cache needs it. If f16 were faster without flash attention, the baseline would have been held back. v2 ran f16 both ways.

Model, mode, context	Decode tok/s, FA on	Decode tok/s, FA off	Request energy, FA off vs on
Llama-1B, 15W, 1K	28.1	25.5	+13.0% ± 3.4
Llama-1B, 15W, 16K	22.8	15.4	+227.6% ± 3.6
Qwen-0.5B, 15W, 1K	60.8	53.1	+19.2% ± 2.6
Qwen-0.5B, 15W, 16K	51.0	28.6	+270.4% ± 2.8
Llama-1B, 7W, 1K	11.8	10.6	+12.8% ± 6.6
Llama-1B, 7W, 16K	10.3	7.0	+211.8% ± 17.2
Qwen-0.5B, 7W, 1K	28.3	23.8	+17.5% ± 1.3
Qwen-0.5B, 7W, 16K	24.9	13.4	+256.5% ± 0.6

10.4 Prefill and decode measured separately (llama-bench)

llama-bench timed prompt processing (pp) and generation (tg) as separate tests, 5 repetitions each (section 8). "tg128 at 16K" = 128 tokens generated after the cache holds 16,384 tokens. The last column repeats the server-run decode J/token (timestamp split) for comparison.

15W

Model, KV	pp16384 tok/s	pp16384 mJ/token	tg128 at 16K tok/s	tg128 at 16K J/token	vs f16	server split J/token
Llama-1B, f16	1,294	5.12	23.2	0.318 ± 0.002	–	0.326
Llama-1B, f16, flash attn off	313	20.41	15.6	0.494 ± 0.001	+55.2%	0.526

Model, KV	pp16384 tok/s	pp16384 mJ/token	tg128 at 16K tok/s	tg128 at 16K J/token	vs f16	server split J/token
Llama-1B, 8-bit	1,250	5.35	18.6	0.434 ± 0.003	+36.5%	0.443
Llama-1B, 4-bit	1,242	5.33	18.0	0.412 ± 0.002	+29.4%	0.419
Qwen-0.5B, f16	2,189	3.11	53.7	0.127 ± 0.000	–	0.136
Qwen-0.5B, f16, flash attn off	497	12.98	29.3	0.249 ± 0.001	+96.1%	0.263
Qwen-0.5B, 8-bit	2,115	3.21	35.4	0.207 ± 0.001	+63.7%	0.215
Qwen-0.5B, 4-bit	2,105	3.20	33.5	0.196 ± 0.001	+54.2%	0.205

7W

Model, KV	pp16384 tok/s	pp16384 mJ/token	tg128 at 16K tok/s	tg128 at 16K J/token	vs f16	server split J/token
Llama-1B, f16	457	6.76	10.4	0.393 ± 0.002	–	0.380
Llama-1B, f16, flash attn off	142	24.85	7.0	0.618 ± 0.002	+57.4%	0.632
Llama-1B, 8-bit	447	7.07	7.2	0.613 ± 0.002	+56.0%	0.593
Llama-1B, 4-bit	440	7.05	6.9	0.580 ± 0.001	+47.7%	0.560
Qwen-0.5B, f16	812	3.93	25.4	0.154 ± 0.001	–	0.151
Qwen-0.5B, f16, flash attn off	230	15.51	13.6	0.318 ± 0.001	+106.1%	0.313
Qwen-0.5B, 8-bit	794	4.05	13.9	0.321 ± 0.002	+108.1%	0.310
Qwen-0.5B, 4-bit	785	4.05	13.0	0.272 ± 0.001	+76.6%	0.263

Net energy (idle subtracted), mean ± 95% CI over 5 repetitions. mJ = millijoules.

10.5 Why: what the extra energy is made of

At 16K tokens	f16	8-bit	4-bit
Llama-1B decode tok/s, 15W	22.8	18.3	17.7
Llama-1B decode power, W gross, 15W	10.49	11.18	10.50
Llama-1B CPU busy / GPU load in decode, 15W	2% / 96%	2% / 96%	2% / 97%
Qwen-0.5B decode tok/s, 15W	51.0	34.0	32.5
Qwen-0.5B decode power, W gross, 15W	9.89	10.38	9.73
Qwen-0.5B CPU busy / GPU load in decode, 15W	3% / 92%	2% / 93%	3% / 94%
Llama-1B decode tok/s, 7W	10.3	7.2	6.8
Llama-1B decode power, W gross, 7W	6.96	7.31	6.87
Llama-1B CPU busy / GPU load in decode, 7W	2% / 98%	2% / 99%	2% / 99%
Qwen-0.5B decode tok/s, 7W	24.9	13.7	12.9
Qwen-0.5B decode power, W gross, 7W	6.78	7.29	6.43
Qwen-0.5B CPU busy / GPU load in decode, 7W	3% / 96%	2% / 97%	2% / 98%

Decode power here is the mean gross VDD_IN power inside the decode window only (section 8). CPU busy is the share of all six CPU cores' time that was busy (/proc/stat); GPU load is the GPU's own busy counter.

Kernel profile. Nsight Systems recorded every GPU kernel while llama-bench filled the cache to 16,384 tokens and then generated 64 tokens (+1 warm-up). Decode attention is the flash-attention kernel launched once per layer per generated token.

Model, KV	Decode attention kernel	µs per call	ms per token	vs f16	Prefill attention, ms	Quantize / dequantize / rotate, ms
Llama-1B, f16	f16 mma (tensor cores)	570	9.1	–	4,075	0

Model, KV	Decode attention kernel	µs per call	ms per token	vs f16	Prefill attention, ms	Quantize / dequantize / rotate, ms
Llama-1B, 8-bit	vec (no tensor cores)	1,201	19.2	+111%	3,967	618
Llama-1B, 4-bit	vec (no tensor cores)	1,309	20.9	+130%	3,963	706
Qwen-0.5B, f16	f16 mma (tensor cores)	151	3.6	–	2,830	0
Qwen-0.5B, 8-bit	vec (no tensor cores)	532	12.8	+253%	2,828	321
Qwen-0.5B, 4-bit	vec (no tensor cores)	589	14.1	+291%	2,825	365

The last column sums the kernels that only exist with a quantized cache: writing new K/V into the cache in 8 or 4 bits, converting the cache back to f16 before each prefill batch, and a Walsh–Hadamard rotation (fwht) that llama.cpp applies to K and Q with a quantized cache. Prefill attention itself takes the same time for all formats because it runs on an f16 copy. With flash attention off, Llama spends 43,450 ms in separate softmax and matrix kernels (against 15,544 ms for everything with FA on).

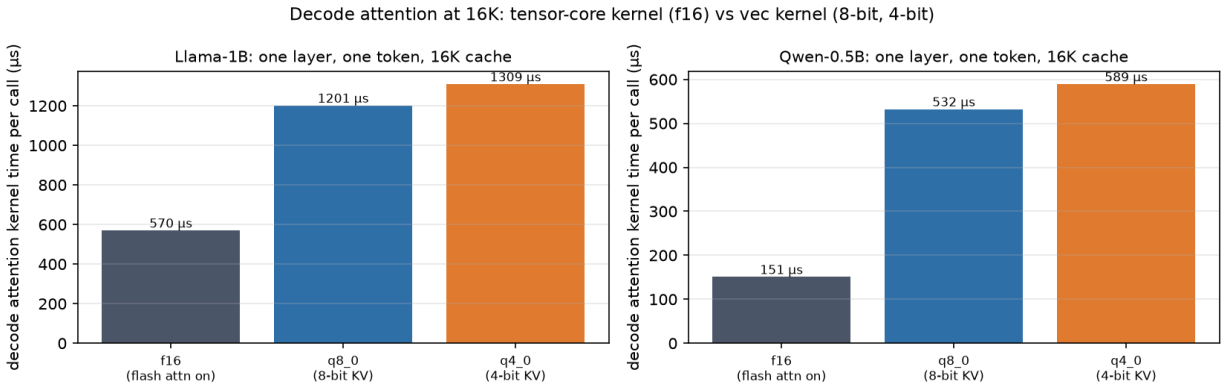


Figure 3. Decode attention kernel time per call at 16K. File: plots_v2/09_kernels.png

Reading. With f16, llama.cpp runs decode attention on a tensor-core (mma) kernel that handles all query heads sharing a KV head together. With q8_0 or q4_0 it switches to the "vec" kernel, which works one query row at a time on the CUDA cores and dequantizes as it reads. It moves fewer bytes but does more instructions, and on this GPU the instructions cost more than the bytes saved. Qwen has 7 query heads per KV head (Llama 4), so the tensor-core kernel helps it more and the switch hurts it more (3.5–3.9× against 2.1–2.3×). This matches the larger energy penalty for Qwen in 10.1–10.2.

Hardware counters (Nsight Compute). The decode attention kernel for one generated token (all layers) at a 16K cache, as a share of the GPU's peak. Nsight Compute locks the clocks to base frequency, so times are longer than in the table above; the shares are what matters.

Model, KV	Kernel	Compute (SM), % of peak	Memory, % of peak	L1 / L2 hit rate	µs per call (clocks locked)
Llama-1B, f16	mma (tensor cores)	10	88	0% / 3%	927
Llama-1B, 4-bit	vec	74	69	66% / 66%	2,001
Llama-1B, 8-bit	vec	65	90	65% / 64%	1,839
Qwen-0.5B, f16	mma (tensor cores)	11	70	3% / 4%	278
Qwen-0.5B, 4-bit	vec	72	63	68% / 78%	872
Qwen-0.5B, 8-bit	vec	64	84	66% / 71%	891

The f16 kernel is memory-bound: it streams the cache at close to the peak memory throughput and leaves the SMs 90% idle. Halving or quartering the bytes should have made decode attention up to 2–4× faster. The vec kernels instead keep the SMs 64–74% busy (unpacking, scaling, one query row at a time) and still run the memory system at 63–90% of peak. Their high L1/L2 hit rates show why: each query head re-reads the K/V block it shares with the other heads of its group, where the f16 kernel reads it once for the whole group. The compressed cache is smaller, but it is read 4 (Llama) or 7 (Qwen) times and unpacked each time.

10.6 Per-rail energy

The INA3221 has three channels. VDD_CPU_GPU_CV feeds the CPU cores, the GPU and the vision accelerators together, so it cannot separate CPU from GPU on its own; the CPU busy counter (10.5) shows the CPU is nearly idle in decode, so changes on this rail are GPU. VDD_SOC feeds the rest of the chip, including the memory controller. The remainder of VDD_IN (memory, regulators, fan, I/O) is shown as "other". Decode energy, net, at 16K, 15W.

Model, KV	VDD_IN, J	CPU+GPU rail, J	SoC rail, J	Other, J	CPU+GPU vs f16	SoC vs f16
Llama-1B, f16	41.7	17.0	10.0	14.7	–	–
Llama-1B, 8-bit	56.7	27.4	11.5	17.8	+61%	+15%
Llama-1B, 4-bit	53.6	27.6	10.4	15.6	+62%	+5%
Qwen-0.5B, f16	17.4	7.6	4.0	5.8	–	–
Qwen-0.5B, 8-bit	27.5	14.5	5.3	7.8	+89%	+32%
Qwen-0.5B, 4-bit	26.2	14.5	4.7	7.0	+90%	+18%

The extra decode energy of a quantized cache lands on the CPU+GPU rail (+60–90%); the SoC rail, which includes the memory controller, changes by 5–32%. If the cost were memory traffic, the SoC and "other" shares would grow instead.

10.7 Concurrency: several sessions at once

N identical requests sent at the same moment to a server with N slots, each slot holding the full context. Energy per generated token over the whole batch (request energy ÷ (N × 128)), net, mean of 3 rounds, and the paired change against f16 with the same N.

4K per session	N = 1	N = 2	N = 4	N = 8
Llama-1B, f16	0.370	0.257	0.222	did not fit
Llama-1B, 8-bit	0.399 (+7.8%)	0.286 (+11.1%)	0.246 (+10.7%)	0.224
Llama-1B, 4-bit	0.393 (+6.3%)	0.282 (+9.6%)	0.242 (+8.7%)	0.221
Qwen-0.5B, f16	0.157	0.129	0.112	0.104
Qwen-0.5B, 8-bit	0.177 (+12.4%)	0.147 (+13.6%)	0.132 (+17.1%)	0.122 (+17.8%)
Qwen-0.5B, 4-bit	0.175 (+11.2%)	0.144 (+11.8%)	0.130 (+16.1%)	0.121 (+16.8%)
16K per session	**N = 1**	**N = 2**	**N = 4**	**N = 8**
Llama-1B, f16	0.959	0.863	0.833	did not fit
Llama-1B, 8-bit	1.100 (+14.7%)	1.010 (+16.9%)	0.985 (+18.3%)	0.974
Llama-1B, 4-bit	1.080 (+12.7%)	0.986 (+14.2%)	0.958 (+15.1%)	0.942
Qwen-0.5B, f16	0.509	0.481	0.469	0.462
Qwen-0.5B, 8-bit	0.603 (+18.4%)	0.591 (+22.7%)	0.577 (+23.0%)	0.575 (+24.6%)
Qwen-0.5B, 4-bit	0.591 (+16.0%)	0.568 (+18.0%)	0.558 (+19.0%)	0.553 (+19.8%)

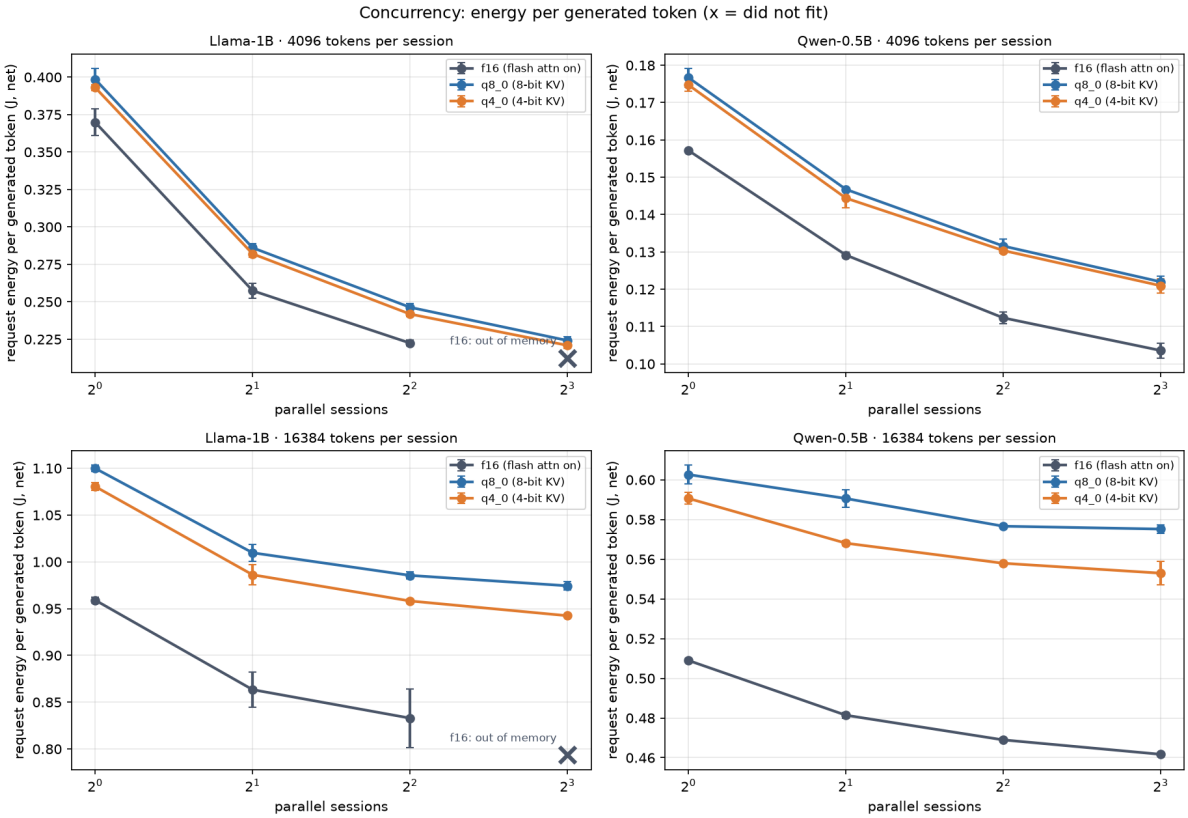


Figure 4. Energy per generated token against the number of sessions. File: plots_v2/07_concurrency.png

Batching lowers the energy per token for every format, because the model weights are read once per step for all sessions. The quantized caches stay more expensive at every N. The one place they win is capacity: Llama-1B with 8 sessions × 16K tokens needs 4.1 GiB of f16 cache, which does not fit next to the weights in 8 GB of shared memory, while 8-bit (2.2 GiB) and 4-bit (1.2 GiB) ran. A separate check confirmed it: llama.cpp's memory check reports "failed to fit params to free device memory", and starting the server anyway froze the board until it rebooted (results/logs/np8_f16_oomcheck.log). (The 4K row for N = 8 is missing for the same reason: one server, sized for 16K per slot, served both contexts.)

10.8 4-bit weights (Q4_K_M)

Same grid as set A with both models' weights in Q4_K_M instead of Q8_0. Smaller weights leave more of each decode step to attention, so if anything the cache format should matter more.

Energy per request, net, change against f16

Context	Llama-1B 8-bit	Llama-1B 4-bit	Qwen-0.5B 8-bit	Qwen-0.5B 4-bit
1K	+3.0% ± 1.5	+2.2% ± 1.4	+5.7% ± 1.6	+4.3% ± 1.1
4K	+8.0% ± 1.6	+7.1% ± 1.4	+11.2% ± 1.5	+10.3% ± 0.9
8K	+11.9% ± 0.4	+10.5% ± 1.0	+16.4% ± 0.7	+13.2% ± 0.8
16K	+15.5% ± 0.3	+13.3% ± 0.3	+18.1% ± 0.1	+15.8% ± 0.4

Decode energy per token, net, change against f16

Context	Llama-1B 8-bit	Llama-1B 4-bit	Qwen-0.5B 8-bit	Qwen-0.5B 4-bit
1K	+2.3% ± 2.3	+1.2% ± 2.3 (n.s.)	+5.3% ± 2.5	+3.3% ± 1.7
4K	+9.5% ± 3.9	+9.1% ± 4.3	+13.1% ± 5.6	+12.0% ± 2.7
8K	+20.8% ± 2.1	+17.6% ± 3.4	+31.0% ± 4.1	+24.4% ± 4.3
16K	+43.1% ± 1.7	+34.9% ± 0.9	+61.1% ± 3.2	+51.1% ± 1.6

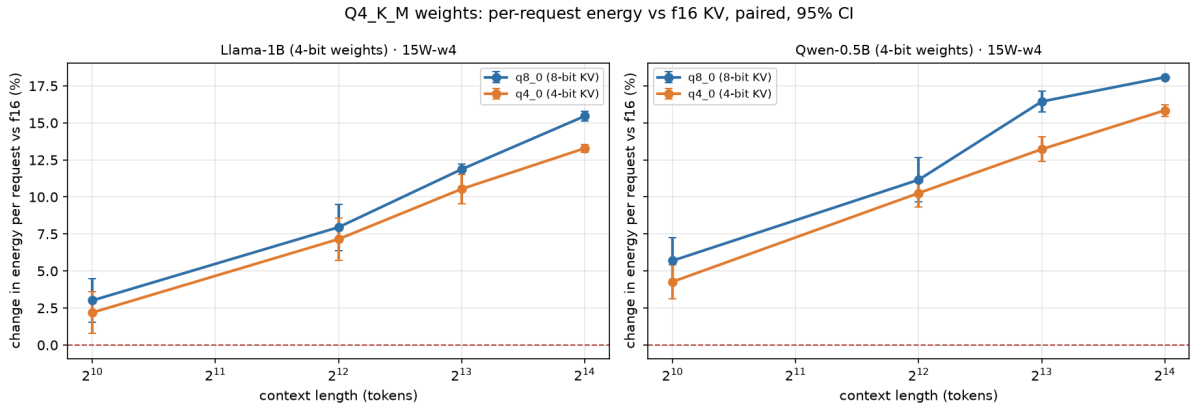


Figure 5. Q4_K_M weights: change in energy per request against f16. File: plots_v2/06_w4_request_change.png

The penalty is the same as with 8-bit weights within a few points. The result is not specific to how the weights are stored.

10.9 Accuracy: perplexity

WikiText-2 test set, llama-perplexity, which scores the second half of each window. At 8K the text is cut into 32 windows of 8,192 tokens; at 2K into 20 windows of 2,048. Lower is better; ± is llama.cpp's standard error. Mixed K/V pairs need the all-quants build (section 5). 16K was planned but does not fit: llama-perplexity keeps the logits of the scored half of a window (8,192 tokens × a 128K–152K-token vocabulary × 4 bytes = 4–5 GB) in memory next to the model, and Linux killed the process.

Model	K	V	Context	Run	Perplexity	vs f16/f16
Llama-1B	f16	f16	2K	v1	12.044 ± 0.229	+0.0%
Llama-1B	f16	q4_0	2K	v2, all-quants	12.097 ± 0.230	+0.4%
Llama-1B	q4_0	f16	2K	v2, all-quants	12.618 ± 0.240	+4.8%
Llama-1B	q4_0	q4_0	2K	v1	12.732 ± 0.242	+5.7%
Llama-1B	q4_1	q4_1	2K	v2, all-quants	12.559 ± 0.238	+4.3%
Llama-1B	q8_0	q8_0	2K	v1	12.049 ± 0.229	+0.0%
Llama-1B	f16	f16	8K	v2	11.066 ± 0.081	+0.0%
Llama-1B	q4_0	q4_0	8K	v2	11.615 ± 0.085	+5.0%
Llama-1B	q8_0	q8_0	8K	v2	11.063 ± 0.081	-0.0%
Qwen-0.5B	f16	f16	2K	v1	13.019 ± 0.253	+0.0%
Qwen-0.5B	f16	q4_0	2K	v2, all-quants	13.060 ± 0.254	+0.3%
Qwen-0.5B	q4_0	f16	2K	v2, all-quants	98.832 ± 2.566	+659.1%
Qwen-0.5B	q4_0	q4_0	2K	v1	100.318 ± 2.599	+670.6%
Qwen-0.5B	q4_0	q4_0	2K	v2, all-quants	100.318 ± 2.599	+670.6%
Qwen-0.5B	q4_0	q8_0	2K	v2, all-quants	96.791 ± 2.508	+643.5%
Qwen-0.5B	q4_1	q4_1	2K	v2, all-quants	96.414 ± 2.472	+640.6%
Qwen-0.5B	q8_0	q4_0	2K	v2, all-quants	13.231 ± 0.258	+1.6%
Qwen-0.5B	q8_0	q8_0	2K	v1	13.185 ± 0.257	+1.3%
Qwen-0.5B	f16	f16	8K	v2	12.491 ± 0.096	+0.0%
Qwen-0.5B	f16	q4_0	8K	v2, all-quants	12.531 ± 0.096	+0.3%
Qwen-0.5B	q4_0	f16	8K	v2, all-quants	156.829 ± 1.697	+1155.6%
Qwen-0.5B	q4_0	q4_0	8K	v2	155.751 ± 1.683	+1146.9%
Qwen-0.5B	q4_0	q4_0	8K	v2, all-quants	155.751 ± 1.683	+1146.9%
Qwen-0.5B	q4_0	q8_0	8K	v2, all-quants	155.635 ± 1.683	+1146.0%
Qwen-0.5B	q4_1	q4_1	8K	v2, all-quants	145.922 ± 1.571	+1068.3%

Model	K	V	Context	Run	Perplexity	vs f16/f16
Qwen-0.5B	q8_0	q4_0	8K	v2, all-quants	12.692 ± 0.097	+1.6%
Qwen-0.5B	q8_0	q8_0	8K	v2	12.649 ± 0.097	+1.3%

Which half breaks Qwen at 4 bits: the keys. At 2K, with 4-bit V and f16 K, Qwen's perplexity is 13.06 (f16: 13.02); with 4-bit K and f16 V it is 98.8. 8-bit K with 4-bit V is also fine (13.23). q4_1, which stores a minimum per block, does not rescue it (96.4), so an offset is not the problem. The likely cause is outlier channels: keys are known to have a few channels with very large values (Qwen2.5 also adds a bias in its K projection), and with one scale per 32 values the small values in a block round to zero. Llama behaves the same way on a smaller scale (4-bit K +5%, 4-bit V +0.4%). At 8K the split gives the same answer: f16 K + 4-bit V 12.53 (f16: 12.49), 4-bit K + f16 V 156.8, q4_1 145.9. For the energy question this means 8-bit K + 4-bit V is the useful configuration to test next: nearly the memory of 4-bit with the accuracy of 8-bit.

10.10 Where compression helps: the memory fit test

One server per format and context, auto-fit off (-fit off) so llama.cpp cannot quietly shrink the context. "did not fit" = the server failed to start (out of memory). 2 rounds.

Model, context	f16	8-bit	4-bit
Llama-1B, 32K	275 J, 18.8 tok/s	319 J, 13.2 tok/s	313 J, 12.9 tok/s
Llama-1B, 64K	761 J, 14.2 tok/s	872 J, 8.1 tok/s	859 J, 8.3 tok/s
Llama-3B (Q4_K_M weights), 16K	342 J, 10.2 tok/s	379 J, 8.1 tok/s	375 J, 7.7 tok/s
Llama-3B (Q4_K_M weights), 32K	did not fit (3.5 GiB cache)	893 J, 5.7 tok/s	887 J, 5.3 tok/s
Llama-3B (Q4_K_M weights), 64K	did not fit (7.0 GiB cache)	did not fit (3.7 GiB cache)	2471 J, 3.2 tok/s

Cells show net energy per request and decode speed, mean of 2 rounds. Llama-1B at 128K was not run: a 131,072-token prompt plus 128 generated tokens exceeds its trained context.

Llama-1B fits 64K in every format, so on the 1B model compression only costs energy. On the 3B model the formats separate: with f16 the cache for 32K (3.5 GiB) does not fit next to the 2 GB of weights, 8-bit reaches 32K and 4-bit 64K. Where both run, 4-bit still uses more energy per request than f16 (at 16K), but it is the only way to serve 64K at all. This is the trade the thesis question asks about: on an 8 GB device, KV quantization buys context length, and pays for it in energy and speed.

10.11 7W against 15W, gross and net

Same requests at both power modes. Positive = 7W uses more energy. Gross includes the idle power that the longer 7W runs keep paying; net does not.

Model, KV, context	Net J, 15W	Net J, 7W	7W vs 15W, net	Gross J, 15W	Gross J, 7W	7W vs 15W, gross
Llama-1B, f16, 1K	33.9	43.0	+27%	50.0	81.0	+62%
Llama-1B, f16, 16K	123.8	154.3	+25%	180.2	300.0	+67%
Llama-1B, 8-bit, 1K	35.4	44.4	+26%	52.0	84.2	+62%
Llama-1B, 8-bit, 16K	143.4	186.1	+30%	205.4	350.6	+71%
Llama-1B, 4-bit, 1K	34.8	44.1	+27%	51.4	83.8	+63%
Llama-1B, 4-bit, 16K	139.3	181.5	+30%	202.7	351.1	+73%
Qwen-0.5B, f16, 1K	13.0	16.8	+29%	20.8	33.1	+60%
Qwen-0.5B, f16, 16K	66.0	79.8	+21%	97.1	156.5	+61%
Qwen-0.5B, 8-bit, 1K	13.6	17.7	+30%	21.8	35.4	+63%
Qwen-0.5B, 8-bit, 16K	77.9	102.1	+31%	113.5	193.0	+70%
Qwen-0.5B, 4-bit, 1K	13.6	17.5	+29%	21.8	35.5	+63%
Qwen-0.5B, 4-bit, 16K	76.6	96.2	+26%	113.0	189.7	+68%

The low-power mode costs energy, both net and gross. At 7W the GPU runs at 408 MHz instead of 625 MHz and decode is 2.0–2.4× slower (Llama f16 at 16K: 22.8 → 10.3 tok/s), while decode power only falls from about 10 W to 7 W. The work therefore costs 21–31% more net energy, and the idle power paid over the longer run raises the gross cost by 60–73%. This confirms v1 ("race to idle") with more rounds. The quantization penalty is somewhat larger at 7W (16K: Llama +21% / +18%, Qwen +28% / +21% for 8-bit / 4-bit, against +13–18% at 15W) because the slower GPU makes the vec kernel's extra compute weigh more. 7W is only worth using under a hard power or thermal limit.

Idle correction. The 7W grid started right after the reboot that the power-mode switch needs, and its start-of-set idle reading (3.68 W) was taken while the board was still settling; the reading at the end of the set was 3.03 W, and the 7W llama-bench set measured 2.92–3.05 W. Net energy in this report therefore subtracts the lower of the start and end idle readings of each set. This changes only the 7W grid (the 15W sets start lower than they end; one differs by 0.01 W). Gross energy is not affected.

11. Data quality

• **Runs.** Every row of every result file is kept, including failures (12 failed runs, all listed below). Analysis uses only runs with status "ok".

• **Spread.** Every paired 95% CI on energy per request in the 15W main grid is within ±3.6 percentage points, most within ±2 (tables in 10.1). The 7W grid is tighter: all within ±1.6 points per request and ±2.5 per decoded token.

• **Idle drift.** Idle VDD_IN at the start and end of the 15W main grid: 3.08 W (start) / 3.34 W (end). The drift (about 0.25 W, fan and board warming up) is small against 9–11 W decode power and affects all formats equally because they are interleaved within each round. 7W: 3.68 W at the start (just after the reboot) and 3.03 W at the end; net energy uses the lower reading (section 10.11).

• **Background load.** v1 ran with the Software Updater active (one core busy). v2 closes it first; the v1 and v2 15W percentages agree within a few points, so it did not change the conclusions.

• **Other software.** An Ollama service (installed on the board before this project) runs in the background in all v1 and v2 runs. It is idle (about 40 MB of memory, no CPU or GPU use), so its draw is part of the measured idle power.

• **Thermals.** GPU temperature and clock are logged after every run. In sets A, D and E the GPU clock was at the 15W maximum (624.75 MHz) after every run and the temperature stayed between 48 and 54 °C. In the 7W grid the clock was at the 7W maximum (408 MHz) after every run, at 45–47 °C. No throttling.

Set, model, format, sessions	Failed runs	Reason
15W-fit3b, Llama-3B (Q4_K_M weights), f16, N = 1	4	out of memory at server start
15W-fit3b, Llama-3B (Q4_K_M weights), q8_0, N = 1	2	out of memory at server start
15W-np, Llama-1B, f16, N = 8	6	out of memory at server start

12. Scope and limitations

• One device (Jetson Orin Nano 8 GB), one engine (llama.cpp 7adf160), two power modes. Other engines with fused low-bit attention kernels (e.g. TensorRT-LLM, MLC) could change the sign of the result; the claim is about this software stack, which is the one most edge users run.

• Small models (0.5B–3B) and greedy decoding with fixed 128-token replies. Longer replies shift the balance further toward decode, where the penalty is largest.

• Energy is measured at the module input (VDD_IN). It includes the fan and regulators but not the carrier board or power adapter.

• Only quantization is covered. Eviction methods (StreamingLLM, SnapKV) are the next experiment.

13. Next steps

• Eviction on the Jetson: StreamingLLM and SnapKV through kvpress (PyTorch for JetPack), same energy method.

- Comparison with the thesis T4 data as the server-side reference point.
- Confirm the main question and claim (section 1) with the supervisors, then start the paper draft.

Appendix A. Reproduction

One-time setup on the Jetson (sudo needed once):

```
sudo apt install -y openssh-server cuda-toolkit-13-2 cmake tmux
bash jetson/setup.sh # llama.cpp + models + data (v1)
bash jetson/setup_v2.sh # llama-bench, all-quants build, Q4_K_M models
```

All v2 experiments (inside tmux):

```
bash jetson/run_v2.sh 15W 2>&1 | tee results/run_v2_15W.log
sudo bash jetson/profile.sh ncu # needs root for GPU counters
sudo nvpmodel -m 1 --force # reboot into 7W
bash jetson/run_v2.sh 7W 2>&1 | tee results/run_v2_7W.log
sudo nvpmodel -m 0 --force # back to 15W
```

On the Mac:

```
rsync -a <user>@<jetson>:~/thesis2/results/ results/
python3 analysis/analyze_v2.py --out plots_v2
python3 analysis/make_report.py
```

Appendix B. Files

File	Contents
results/kv_20261019_190621.jsonl	15W: v1 main grid, 15W (+ .meta.json: settings, idle power, versions)
results/kv_20261019_194427.jsonl	7W: v1 main grid, 7W (+ .meta.json: settings, idle power, versions)
results/kv_20261026_211242.jsonl	15W-v2: set A, 15W (+ .meta.json: settings, idle power, versions)
results/kv_20261026_221933.jsonl	15W-np: set D, concurrency (+ .meta.json: settings, idle power, versions)
results/kv_20261026_234121.jsonl	15W-w4: set E, Q4_K_M weights (+ .meta.json: settings, idle power, versions)
results/kv_20261027_004802.jsonl	15W-fit1b: set G, Llama-1B (+ .meta.json: settings, idle power, versions)
results/kv_20261027_010512.jsonl	15W-fit3b: set G, Llama-3B (+ .meta.json: settings, idle power, versions)
results/kv_20261027_022931.jsonl	7W-v2: set A, 7W (+ .meta.json: settings, idle power, versions)
results/pptg_20261026_215258.jsonl	set B, llama-bench, 15W (+ .meta.json)
results/pptg_20261027_035440.jsonl	set B, llama-bench, 7W (+ .meta.json)
results/ppl_*.csv	perplexity (v1 at 2K; v2 at 8K and the K/V split; the 16K attempts are the rows without a value)
results/prof/	nsys kernel summaries (*_cuda_gpu_kern_sum.csv) and ncu counters (*_ncu.csv)
results/logs/	llama-server log of every server start
plots_v2/stats.json	every number in this report, with CIs
analysis/analyze_v2.py, make_report.py, report_content.py	statistics, figures and this PDF

Appendix C. v1 per-round values: Qwen-0.5B, 7W, 16K

Format	Round	Decode J/token, net	Decode tok/s	Request energy, net J	GPU °C at end
f16	1	0.134	24.9	70.1	45.9
f16	2	0.135	24.8	70.0	46.0
f16	3	0.135	24.8	70.0	46.2
q8_0	1	0.281	13.6	90.4	46.1
q8_0	2	0.281	13.6	90.6	45.9
q8_0	3	0.282	13.6	90.4	45.8
q4_0	1	0.235	12.8	84.6	46.0
q4_0	2	0.233	12.8	84.4	45.8
q4_0	3	0.234	12.9	84.5	45.8

The review asked whether this cell (decode energy roughly doubled by 8-bit) was an outlier. The rounds agree closely, so it is a stable result of the 7W mode, not noise.

Appendix D. Glossary

Term	Meaning
KV cache	The keys and values of every token read so far, kept so each new token only computes its own. Grows linearly with context.
Prefill / decode	Reading the prompt (all tokens at once) / generating the reply one token at a time.
Flash attention (FA)	Attention computed in one fused kernel without storing the full score matrix.
Tensor cores	GPU units for small matrix multiplies; much faster than ordinary CUDA cores for that.
mma / vec kernel	llama.cpp's tensor-core flash-attention kernel / its one-row CUDA-core kernel.
Net / gross energy	Energy with idle power subtracted / energy the module actually drew.
Paired change, 95% CI	Mean of per-round ratios against f16 from the same round, with the interval that contains the true mean with 95% confidence.
n.s.	Not significant: the CI includes zero.
Perplexity	How surprised the model is by real text; lower is better.
Q8_0, Q4_K_M	Weight formats: 8-bit; 4-bit with per-block scales (K-quant, "medium").